TSD

TECHNICAL SPECIFICATION DOCUMENT

Employee Onboarding Dashboard

Attribute	Value
Document Version	1.0
Date	[Date]
Author	[Author]
Status	Draft — for review
Reference	Companion to BRD-EOD-001

1. Purpose, Scope, and Status

This Technical Specification Document (TSD) provides the authoritative engineering specification for the Employee Onboarding Dashboard (EOD). While the companion Business Requirements Document (BRD-EOD-001) details the operational problem, user personas, and functional capabilities (the "what"), this document defines the concrete architecture, integration protocols, database schemas, computational models, security boundaries, and infrastructure operational requirements (the "how").

A warning the reader must not skip

1. **Strict AI Boundary**: Generative AI is architecturally restricted to a single capability: converting a manager's free-text input into an unapproved draft task list. It does not recommend contacts, select compliance certifications, summarise progress, or query static documentation. No AI output may be made visible to a joinee without explicit, audited manager approval.
2. **Manual Path Resilience**: AI task drafting is an optional accelerator. All operational workflows (plan authoring, task management, certification assignment, progress tracking) must function fully in the event of total LLM provider degradation or failure.

2. Requirement Traceability (BRD → TSD)

Technical requirement identifiers are partitioned into subsystems:

- **TR-1.x**: Identity, User Mapping, and Access Control
- **TR-2.x**: Plan Lifecycle, Task Management, and Date Calculations
- **TR-3.x**: AI Orchestration, Validation, Caching, and Run Auditing
- **TR-4.x**: Certifications and Connection Records
- **TR-5.x**: Information Hub and Content Seeding
- **TR-6.x**: Progress Computations, Dashboards, and Joinee Interactions
- **TR-7.x**: Scheduled Jobs, Notifications, and Escalations
- **TR-8.x**: Core Audit Trail and Lifecycle Completion

BRD ID	Technical Requirement(s)	Phase
BR-01	TR-1.1, TR-1.2	Phase 1 — Foundation
BR-02	TR-1.3, TR-1.4	Phase 1 — Foundation
BR-03	TR-2.1, TR-2.2	Phase 3 — Manual Core
BR-04	TR-3.1, TR-3.2	Phase 4 — AI Task Generation
BR-05	TR-3.3, TR-3.4	Phase 4 — AI Task Generation
BR-06	TR-3.5	Phase 4 — AI Task Generation
BR-07	TR-3.6, TR-3.7	Phase 4 — AI Task Generation
BR-08	TR-3.8	Phase 4 — AI Task Generation
BR-09	TR-3.9	Phase 4 — AI Task Generation
BR-10	TR-2.3	Phase 3 — Manual Core
BR-11	TR-2.4	Phase 3 — Manual Core
BR-12	TR-2.5	Phase 3 — Manual Core

BRD ID	Technical Requirement(s)	Phase
BR-13	TR-2.6	Phase 3 — Manual Core
BR-14	TR-4.1, TR-4.2	Phase 3 — Manual Core
BR-15	TR-4.3	Phase 3 — Manual Core
BR-16	TR-4.4	Phase 3 — Manual Core
BR-17	TR-6.1	Phase 3 — Manual Core
BR-18	TR-6.2, TR-8.1	Phase 3 — Manual Core
BR-19	TR-4.5, TR-4.6	Phase 3 — Manual Core
BR-20	TR-4.7	Phase 3 — Manual Core
BR-21	TR-6.3, TR-6.4	Phase 3 — Manual Core
BR-22	TR-6.5	Phase 3 — Manual Core
BR-23	TR-5.1	Phase 2 — Information Hub
BR-24	TR-5.2, TR-5.3	Phase 2 — Information Hub
BR-25	TR-5.4	Phase 2 — Information Hub
BR-26	TR-5.5	Phase 2 — Information Hub
BR-27	TR-7.1, TR-7.2	Phase 5 — Notifications and Hardening
BR-28	TR-7.3	Phase 5 — Notifications and Hardening
BR-29	TR-3.10	Phase 4 — AI Task Generation
BR-30	TR-8.2	Phase 5 — Notifications and Hardening
BR-31	TR-8.3	Phase 3 — Manual Core
BR-32	TR-6.6	Phase 3 — Manual Core

3. Architecture

3.1 Logical Architecture

The Employee Onboarding Dashboard is architected as a modular Spring Boot 3.x monolith paired with a React Single-Page Application (SPA), bounded by PostgreSQL 16 for relational persistence, Redis for caching/session state, and S3-compatible Object Storage for document assets.

All inbound HTTPS traffic terminates at an API Gateway / Reverse Proxy, which enforces rate limits and routes traffic either to static SPA bundles or the Spring Boot core API. Authentication is governed by OpenID Connect (OIDC) through corporate identity providers. The backend enforces strict role-based access control (RBAC) and data isolation using security interceptors.

Asynchronous notification processing and batch health checks are governed via Spring Scheduler. Generative AI interactions occur through an isolated **`AI Orchestration Module`** backed by Redis caching, external LLM client interfaces, and schema validation interceptors.

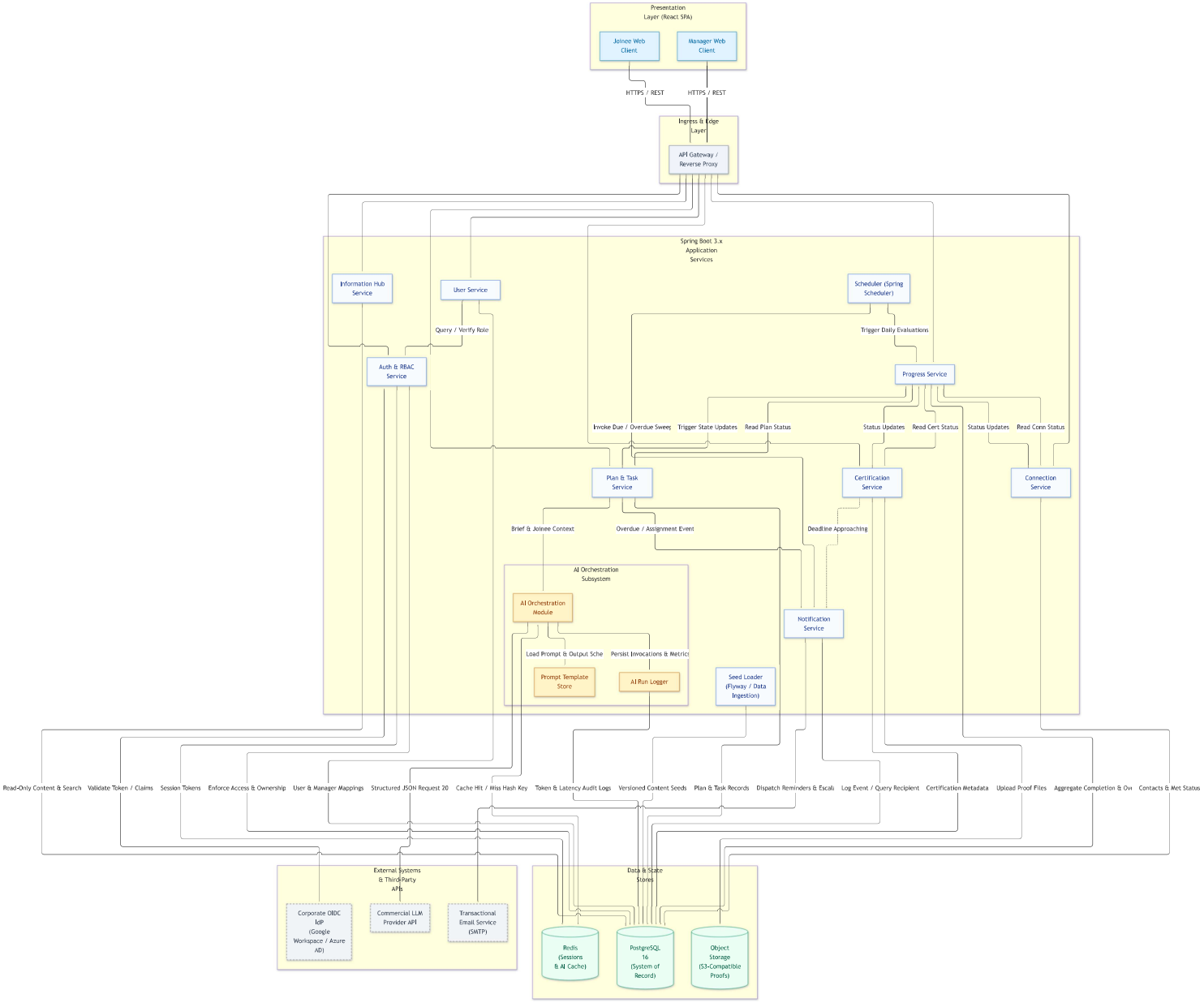


Figure 1 — Architecture

3.2 Components

ID	Component	Responsibility	Technology
C-01	Manager Web Client	Plan authoring, AI brief submission, task review grid, certification/connection input, and progress tracking.	React 18, TypeScript, Vite, Tailwind CSS
C-02	Joinee Web Client	Active plan consumption, task execution updates, proof uploads, contact tracking, Information Hub search.	React 18, TypeScript, Vite, Tailwind CSS
C-03	API Gateway / Proxy	TLS termination, HTTP security headers, rate limiting, static asset proxying.	Nginx / Cloud Reverse Proxy

ID	Component	Responsibility	Technology
C-04	Auth & RBAC Service	OIDC JWT verification, token parsing, role resolution (`MANAGER`, `JOINEE`), server-side relationship validation.	Spring Security 6, Nimbus JOSE JWT
C-05	User Service	User metadata management, manager-to-joinee reporting line validation.	Spring Data JPA, Java 21
C-06	Plan & Task Service	Plan state transitions (`DRAFT`, `ACTIVE`, `COMPLETED`), task CRUD, due-date synthesis from offsets.	Spring Data JPA, Java 21
C-07	AI Orchestration Module	Prompt injection assembly, LLM API dispatch, fallback orchestration, schema validation, Redis cache lookup.	Java 21, Spring WebClient, Jackson
C-08	Prompt Template Store	Externalised, version-controlled repository containing base prompt templates and structural schemas.	Filesystem / Classpath Resources
C-09	AI Run Logger	Audit logger for LLM invocations recording prompt meta, tokens, raw payload, latency, validation, and acceptance.	Spring Data JPA, PostgreSQL
C-10	Certification Service	Certification catalog CRUD, mandatory item enforcement, document upload management.	Spring Data JPA, AWS S3 SDK (Java)
C-11	Connection Service	Management and tracking of unstructured contact lists and interaction milestones (`Pending`, `Scheduled`, `Met`).	Spring Data JPA, Java 21
C-12	Information Hub Service	Read-only access to categorised organizational reference content; full-text keyword search.	Spring Data JPA, PostgreSQL tsvector
C-13	Seed Loader	Cold-start and migration ingestion of static Information Hub data from versioned JSON/YAML artifacts.	Flyway, Jackson YAML
C-14	Progress Service	Real-time calculation of completion metrics, overdue status aggregations, and dashboard views.	Spring Data JPA, Native SQL queries
C-15	Notification Service	Composition and dispatch of automated email reminders and manager escalations.	Spring Mail, JavaMailSender, SMTP
C-16	Scheduler	Cron-based background evaluation of due dates and escalation trigger dispatching.	Spring Scheduler (`@Scheduled`)
C-17	Relational Database	System of record for all entities, audit logs, and reference items.	PostgreSQL 16

ID	Component	Responsibility	Technology
C-18	Cache / Session Store	Key-value store for OIDC session caching and SHA-256 context hash caching for AI responses.	Redis 7
C-19	Object Storage	Secure S3-compliant bucket for immutable storage of certification proof files.	S3 / MinIO

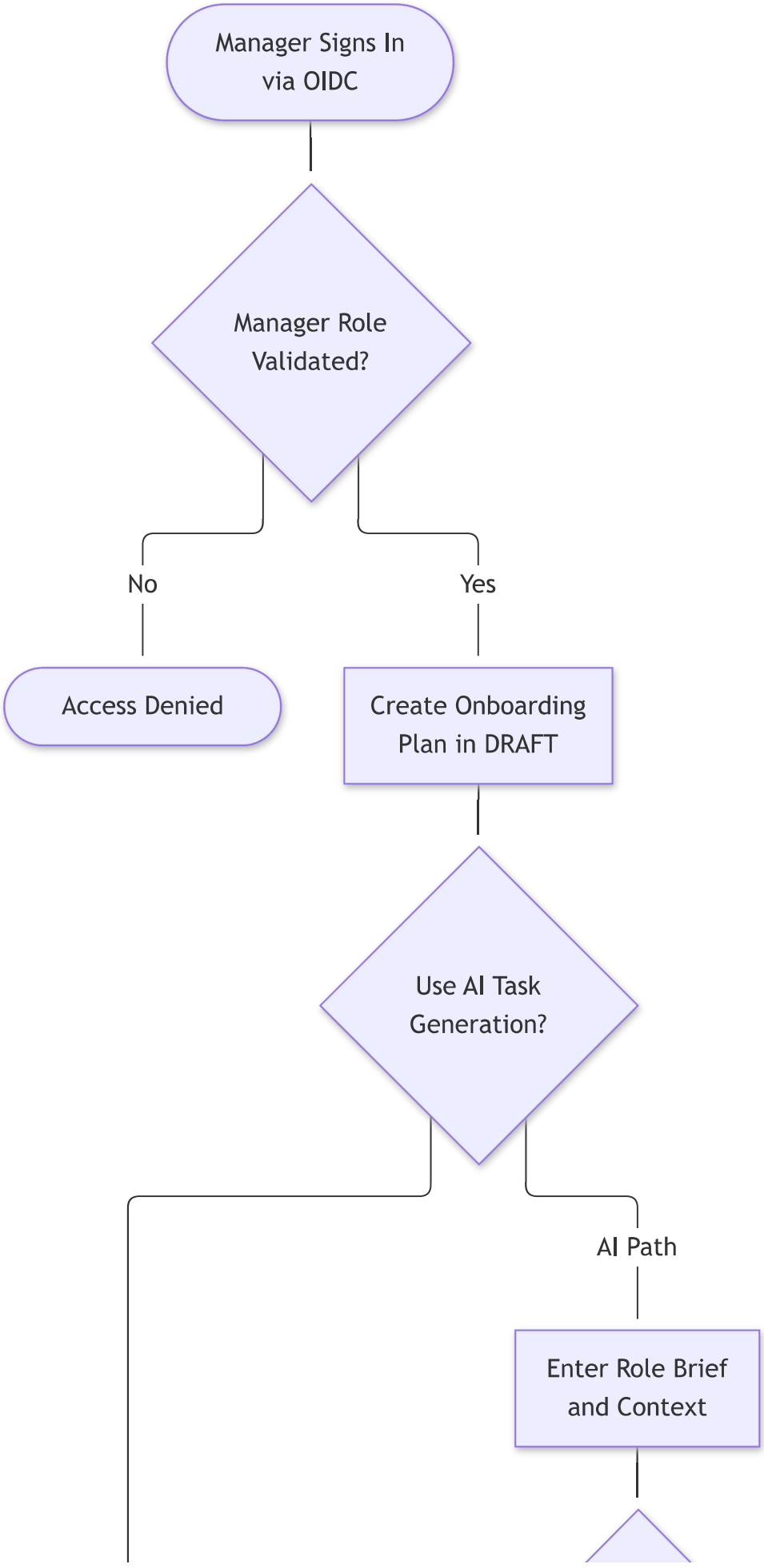
3.3 The Load-Bearing Architectural Rule

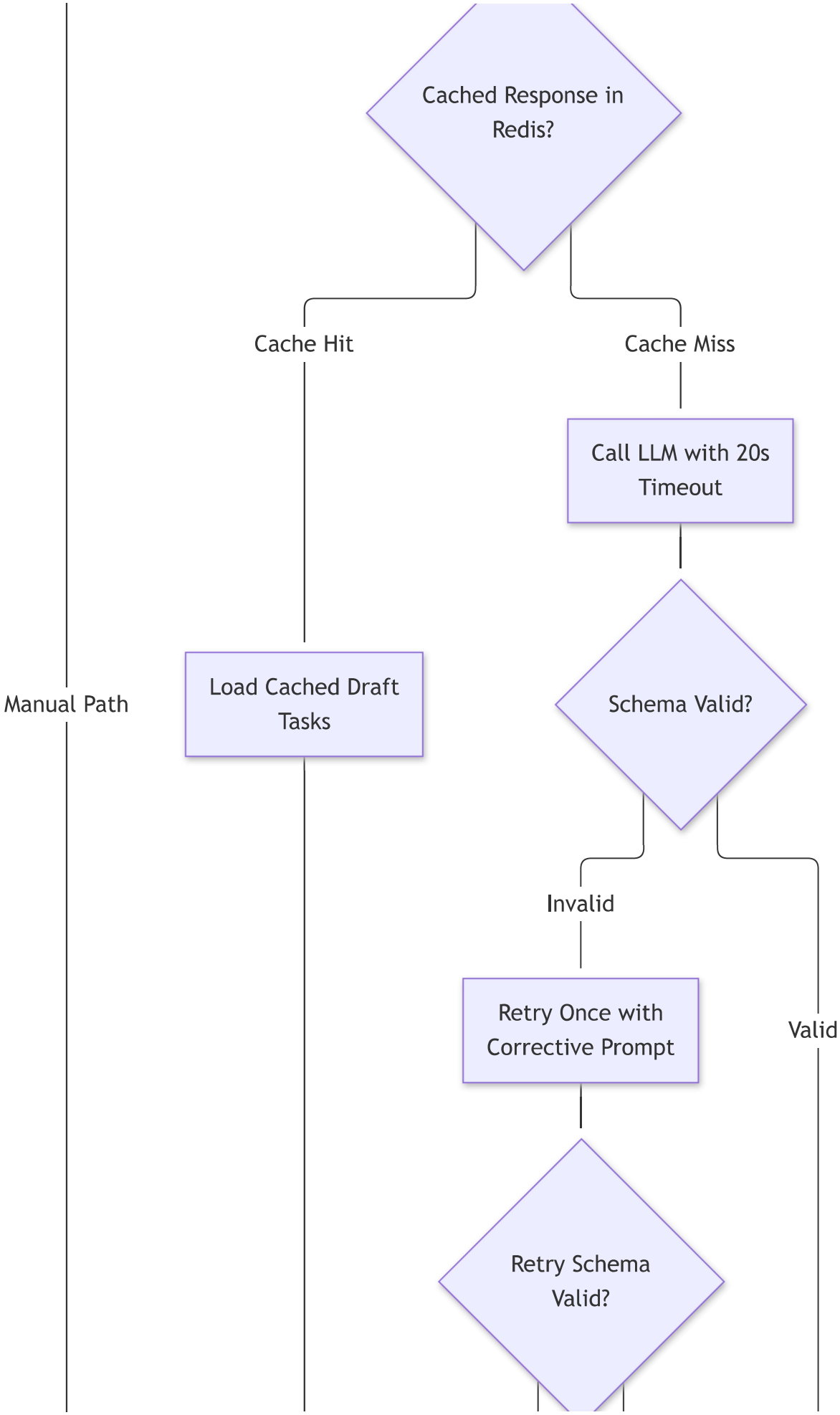
Managerial Boundary and Server-Derived Tenancy: An onboarding plan in `DRAFT` status and its constituent generated tasks are strictly inaccessible to the assigned `JOINEE` until explicitly transitioned to `ACTIVE` by an authenticated `MANAGER`. All query filters and mutation permissions must be derived strictly from the authenticated JWT principal context—client-supplied user or tenant identifiers are never trusted for ownership evaluation. Violations discovered in code review (e.g., passing a mutable `manager_id` or `joinee_id` in request payloads to bypass context checks) must be rejected immediately.

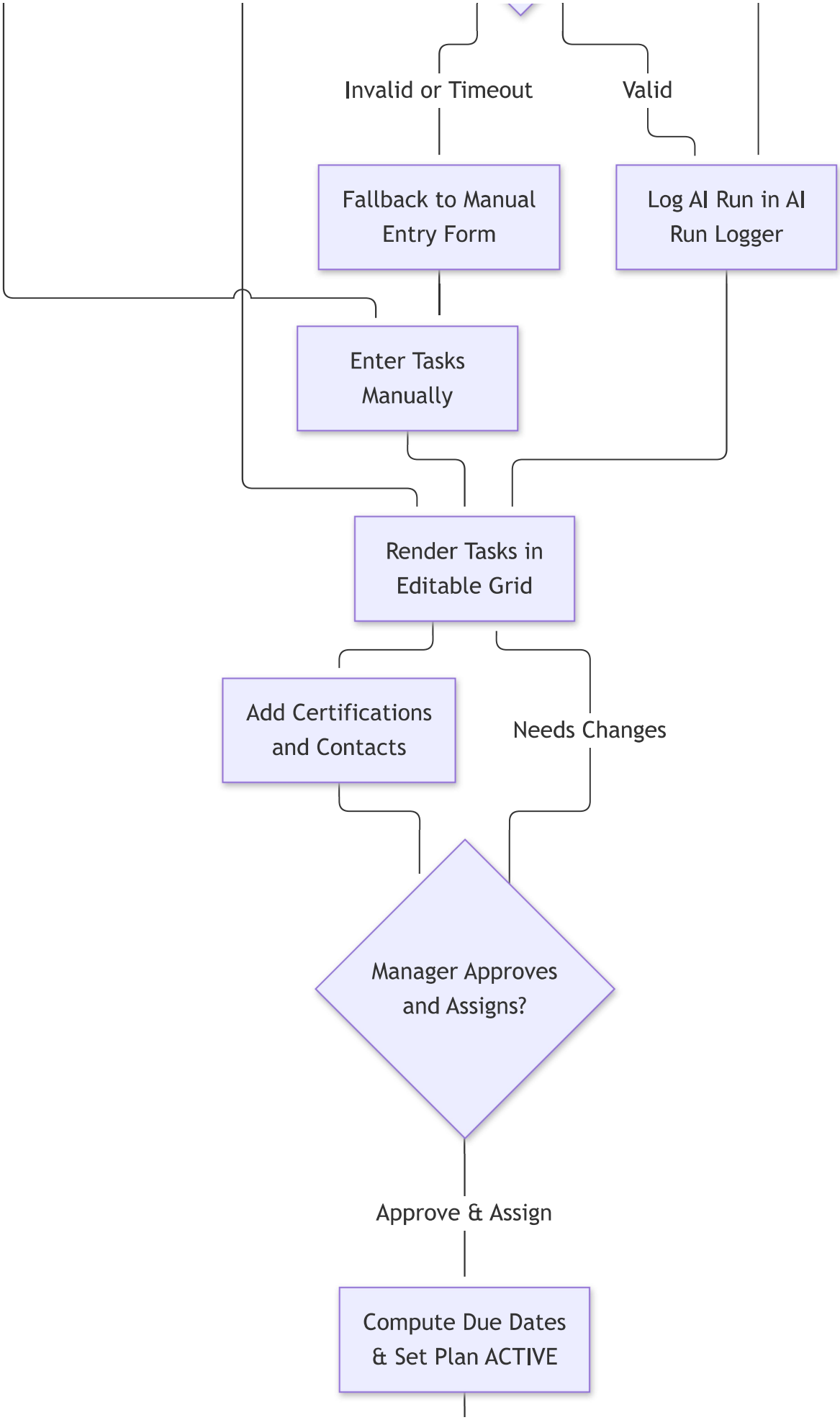
3.4 Data Flow

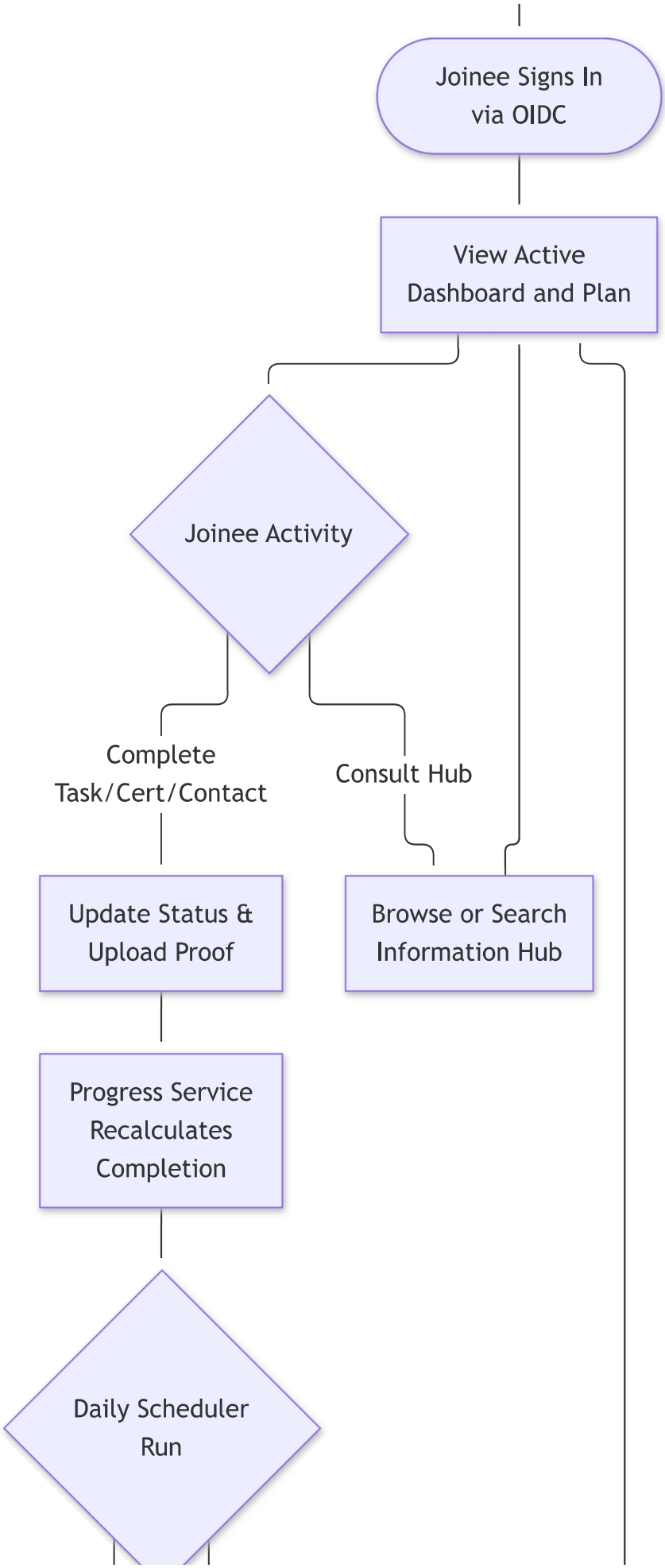
Step	From	To	Mechanism	Frequency
1	Web Client	Auth & RBAC Service	OIDC Authorization Code Flow + PKCE / Bearer JWT	Per user session init
2	Manager Web Client	Plan & Task Service	HTTPS POST `/api/v1/plans`	On demand
3	Manager Web Client	AI Orchestration Module	HTTPS POST `/api/v1/plans/{id}/ai-draft`	On demand
4	AI Orchestration Module	Redis	Jedis / Lettuce `GET key = hash(brief+context)`	Per AI request
5	AI Orchestration Module	External LLM API	HTTPS POST (20s timeout, TLS 1.3)	On demand (cache miss)
6	AI Orchestration Module	AI Run Logger	Async Database Insert (`ai_run_log`)	Every invocation attempt
7	Manager Web Client	Plan & Task Service	HTTPS PUT `/api/v1/plans/{id}/assign`	On demand

Step	From	To	Mechanism	Frequency
8	Joinee Web Client	Progress Service	HTTPS GET `/api/v1/joinee/dashboard`	Per joinee dashboard load
9	Joinee Web Client	Information Hub Service	HTTPS GET `/api/v1/info-hub?search={q}`	On demand
10	Joinee Web Client	Certification Service	HTTPS POST (Multipart `file` to S3 + metadata)	On demand
11	Scheduler	Notification Service	Direct Spring Bean invocation	Daily cron (04:00 UTC)
12	Notification Service	SMTP Gateway	SMTPS (Port 587/465, TLS)	Daily batch trigger


Manager Signs In via OIDC
Manager Role Validated?
No
Yes
Access Denied
Create Onboarding Plan in DRAFT
Use AI Task Generation?
AI Path
Enter Role Brief and Context


Cached Response in Redis?
Cache Hit
Cache Miss
Call LLM with 20s Timeout
Load Cached Draft Tasks
Schema Valid?
Manual Path
Invalid
Retry Once with Corrective Prompt
Valid
Retry Schema Valid?


Invalid or Timeout
Valid
Fallback to Manual Entry Form
Log AI Run in AI Run Logger
Enter Tasks Manually
Render Tasks in Editable Grid
Add Certifications and Contacts
Needs Changes
Manager Approves and Assigns?
Approve & Assign
Compute Due Dates & Set Plan ACTIVE


Joinee Signs In via OIDC
View Active Dashboard and Plan
Joinee Activity
Complete Task/Cert/Contact
Consult Hub
Update Status & Upload Proof
Browse or Search Information Hub
Progress Service Recalculates Completion
Daily Scheduler Run

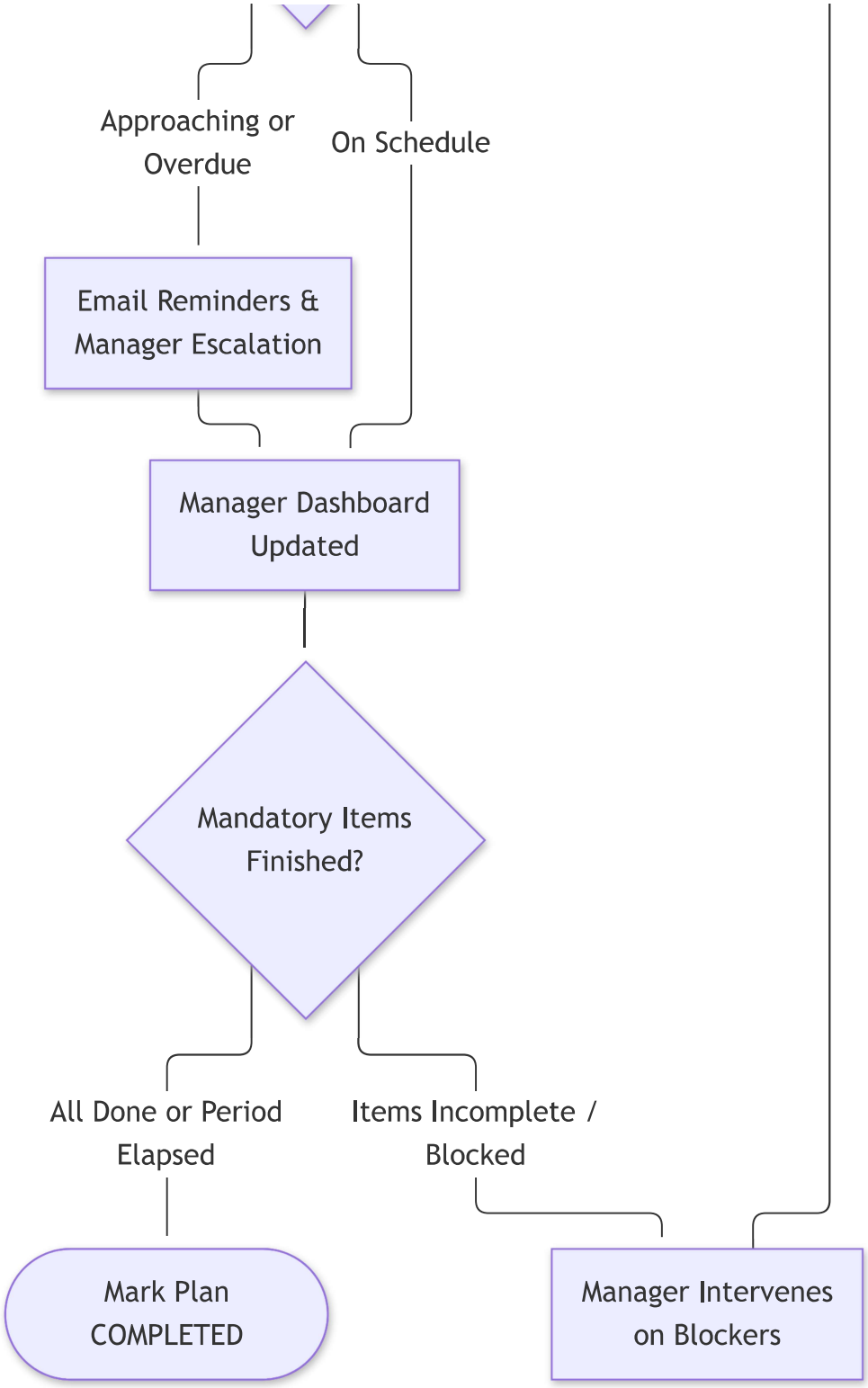


Figure 2 — Process Flow

4. Environments

Environment	Source / Deployment Trigger	Purpose
Development (`dev`)	Continuous Deployment on merge to `develop` branch.	Integrated backend/frontend validation, ephemeral seed ingestion testing, local/mock LLM execution.
Staging / Test (`stage`)	Tagged release candidate (`rc-*`) deployment via CI/CD.	UAT, Testcontainers integration testing, live OIDC sandbox validation, LLM golden prompt evaluation.
Production (`prod`)	Manual sign-off promotion of release tag (`v*.*.*`).	Live multi-user operations, immutable Flyway migrations, high-availability PostgreSQL and Redis backing.

5. Source/System Integration

5.1 Corporate Identity Provider (OIDC)

- **Protocol**: OpenID Connect Core 1.0 (OAuth 2.0 Authorization Code Flow with PKCE).
- **Scope**: `openid profile email`.
- **Claim Extraction**:
  - `sub`: Unique identity provider subject identifier.
  - `email`: Corporate email address.
  - `roles` / `groups`: Token claim mapping to application roles (`ROLE_MANAGER`, `ROLE_JOINEE`).

5.2 External Commercial LLM API

- **Protocol**: HTTPS REST endpoint using JSON payloads.
- **Data Boundary Enforcement**: Payloads are scrubbed of PII. Permitted context is restricted to:
  - `job_title` (string)
  - `team` (string)
  - `start_date` (ISO-8601 date)
  - `plan_duration_days` (integer)
  - `manager_brief` (string)

- **Rate and Cost Control**:
  - Inbound prompt requests hashed via SHA-256 (`SHA-256(job_title + team + plan_duration + brief)`).
  - Cache TTL: 7 days in Redis.
  - Per-plan generation rate-limit: 3 generation requests per plan per hour.
  - Provider SLA Timeout: 20 seconds hard timeout.

5.3 Object Storage (S3 API Compatible)

- **Protocol**: S3 Signature Version 4.
- **Bucket Layout**: `eod-proof-documents/{joinee_id}/{plan_id}/{certification_id}_{uuid}.{ext}`.
- **Validation Interceptor**: File magic-number inspection (allowlist: `application/pdf`, `image/png`, `image/jpeg`), size capped at 10 MiB, ClamAV antivirus daemon scanning prior to storage persist.

---

6. Data Model

6.1 Database Schema (PostgreSQL 16)

```
-- Schema DDL: Core Tables

CREATE TYPE user_role AS ENUM ('MANAGER', 'JOINEE');
CREATE TYPE plan_status AS ENUM ('DRAFT', 'ACTIVE', 'COMPLETED');
CREATE TYPE task_category AS ENUM (
    'TECHNICAL_SETUP', 'ACCESS_AND_TOOLS', 'COMPLIANCE',
    'TEAM_AND_PROCESS', 'ROLE_RAMP_UP', 'ADMIN'
);
CREATE TYPE task_priority AS ENUM ('HIGH', 'MEDIUM', 'LOW');
CREATE TYPE task_status AS ENUM ('TO_DO', 'IN_PROGRESS', 'DONE', 'BLOCKED');
CREATE TYPE connection_status AS ENUM ('PENDING', 'SCHEDULED', 'MET');

CREATE TABLE app_user (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    oidc_sub VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    full_name VARCHAR(255) NOT NULL,
    role user_role NOT NULL,
    manager_id UUID REFERENCES app_user(id),
    job_title VARCHAR(100),
    team VARCHAR(100),
    start_date DATE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE onboarding_plan (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    joinee_id UUID NOT NULL REFERENCES app_user(id),
    manager_id UUID NOT NULL REFERENCES app_user(id),
    status plan_status NOT NULL DEFAULT 'DRAFT',
    duration_days INT NOT NULL DEFAULT 30,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    approved_at TIMESTAMP WITH TIME ZONE,
    completed_at TIMESTAMP WITH TIME ZONE
);

CREATE TABLE plan_task (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plan_id UUID NOT NULL REFERENCES onboarding_plan(id) ON DELETE CASCADE,
    title VARCHAR(100) NOT NULL,
    description VARCHAR(500),
    category task_category NOT NULL,
    week_no INT NOT NULL CHECK (week_no BETWEEN 1 AND 4),
    day_offset INT NOT NULL CHECK (day_offset BETWEEN 0 AND 30),
    due_date DATE,
    priority task_priority NOT NULL DEFAULT 'MEDIUM',
    status task_status NOT NULL DEFAULT 'TO_DO',
    subtasks JSONB DEFAULT '[]'::jsonb,
    joinee_notes TEXT,
    completed_at TIMESTAMP WITH TIME ZONE,
    is_ai_generated BOOLEAN DEFAULT FALSE,
    is_modified_by_manager BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE certification (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plan_id UUID NOT NULL REFERENCES onboarding_plan(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    provider VARCHAR(255) NOT NULL,
    url TEXT NOT NULL,
    deadline DATE NOT NULL,
    is_mandatory BOOLEAN NOT NULL DEFAULT TRUE,
    is_completed BOOLEAN NOT NULL DEFAULT FALSE,
    proof_document_key VARCHAR(512),
    completed_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE connection_contact (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plan_id UUID NOT NULL REFERENCES onboarding_plan(id) ON DELETE CASCADE,
    contact_name VARCHAR(255) NOT NULL,
    contact_email VARCHAR(255) NOT NULL,
    contact_title VARCHAR(255) NOT NULL,
    reason TEXT NOT NULL,
    status connection_status NOT NULL DEFAULT 'PENDING',
    met_date DATE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE info_resource (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category VARCHAR(100) NOT NULL,
    title VARCHAR(255) NOT NULL,
    body_md TEXT NOT NULL,
    contact_email VARCHAR(255),
    link_url TEXT,
    display_order INT NOT NULL DEFAULT 0,
    last_reviewed_date DATE NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    tsv_search TSVECTOR GENERATED ALWAYS AS (
        to_tsvector('english', coalesce(title, '') || ' ' || coalesce(body_md, ''))
```

```
    ) STORED
);
CREATE INDEX idx_info_resource_search ON info_resource USING GIN(tsv_search);

CREATE TABLE ai_run_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plan_id UUID REFERENCES onboarding_plan(id),
    manager_id UUID NOT NULL REFERENCES app_user(id),
    prompt_version VARCHAR(50) NOT NULL,
    model_identifier VARCHAR(100) NOT NULL,
    input_brief TEXT NOT NULL,
    raw_output JSONB,
    validation_passed BOOLEAN NOT NULL,
    validation_errors TEXT,
    prompt_tokens INT,
    completion_tokens INT,
    total_tokens INT,
    latency_ms BIGINT NOT NULL,
    is_accepted BOOLEAN DEFAULT FALSE,
    is_edited BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE audit_event (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id UUID NOT NULL REFERENCES app_user(id),
    entity_type VARCHAR(100) NOT NULL,
    entity_id UUID NOT NULL,
    action VARCHAR(100) NOT NULL,
    payload_diff JSONB,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

6.2 Excluded by Construction

- **Employee Personal Data in AI Pipeline**: `app_user.full_name`, `app_user.email`, and internal database primary keys (`UUID`) are stripped before generating model prompts.
- **Dynamic User References for Connections**: `connection_contact` deliberately stores free-text strings for `contact_name` and `contact_email` without Foreign Key constraints to `app_user`,

allowing non-system external contacts to be assigned.

- **Document Payload Storage**: PostgreSQL never stores binaries; `proof_document_key` stores only immutable S3 object keys.

---

7. Core Logic and Metric Implementation

7.1 Due-Date Synthesis

```
def calculate_due_date(start_date: datetime.date, day_offset: int) -> datetime.date:
    """
    Computes absolute due date based on start date and offset.
    Offset 0 indicates start date itself.
    """
    if start_date is None or day_offset is None:
        raise ValueError("start_date and day_offset must not be null")
    if not (0 <= day_offset <= 30):
        raise ValueError("day_offset must be between 0 and 30")
    return start_date + datetime.timedelta(days=day_offset)
```

7.2 Progress and Completion Aggregation Engine

```
def compute_dashboard_metrics(plan_id: UUID, current_date: datetime.date) -> dict:
    tasks = db.query(PlanTask).filter(PlanTask.plan_id == plan_id).all()
    certs = db.query(Certification).filter(Certification.plan_id == plan_id).all()
    connections = db.query(ConnectionContact).filter(ConnectionContact.plan_id == plan

    total_tasks = len(tasks)
    completed_tasks = sum(1 for t in tasks if t.status == TaskStatus.DONE)

    total_mandatory_certs = sum(1 for c in certs if c.is_mandatory)
    completed_mandatory_certs = sum(1 for c in certs if c.is_mandatory and c.is_comple

    total_connections = len(connections)
    met_connections = sum(1 for conn in connections if conn.status == ConnectionStatus

    # Calculate Checklist Task Completion Percentage
```

```
    # Edge case: If plan has no tasks assigned, completion is defined as NULL (never 0
    task_completion_pct = (
        round((completed_tasks / total_tasks) * 100.0, 2)
        if total_tasks > 0 else None
    )

    # Overdue Task Count: Incomplete tasks where current_date > due_date
    overdue_task_count = sum(
        1 for t in tasks
        if t.status != TaskStatus.DONE and t.due_date is not None and current_date > t
    )

    # Overdue Certification Count: Incomplete mandatory certifications past deadline
    overdue_cert_count = sum(
        1 for c in certs
        if not c.is_completed and current_date > c.deadline
    )

    total_overdue_items = overdue_task_count + overdue_cert_count

    return {
        "task_completion_percentage": task_completion_pct,
        "completed_tasks": completed_tasks,
        "total_tasks": total_tasks,
        "completed_mandatory_certs": completed_mandatory_certs,
        "total_mandatory_certs": total_mandatory_certs,
        "met_connections": met_connections,
        "total_connections": total_connections,
        "total_overdue_items": total_overdue_items
    }
```

7.3 AI Quality Metric Formulas

- **AI Acceptance Rate**: $$\text{Acceptance Rate} = \frac{\sum \text{Generated Tasks Assigned Unmodified or Modified}}{\sum \text{Total Generated Tasks in Run}} \times 100$$
- **AI Task Edit Rate**: $$\text{Task Edit Rate} = \frac{\sum \text{Generated Tasks Where } \texttt{is_modified_by_manager} = \text{TRUE}}{\sum \text{Assigned AI Tasks}} \times 100$$

- **First-Attempt Schema Failure Rate**: $$\text{Schema Failure Rate} = \frac{\sum \text{Runs with Attempt 1 Failure}}{\sum \text{Total Unique AI Prompt Invocations}} \times 100$$

---

8. Interface Specifications

8.1 API Contract Directory

Method	Path	Description	Access Role
`GET`	`/api/v1/auth/me`	Resolve identity, permissions, and profile.	Authenticated
`GET`	`/api/v1/manager/joinees`	Fetch list of mapped joinees with progress summary.	`MANAGER`
`POST`	`/api/v1/manager/joinees`	Register new joinee record under authenticated manager.	`MANAGER`
`POST`	`/api/v1/plans`	Create onboarding plan in `DRAFT` status.	`MANAGER`
`POST`	`/api/v1/plans/{id}/ai-draft`	Generate draft task array from brief via LLM.	`MANAGER`
`PUT`	`/api/v1/plans/{id}/tasks`	Bulk update/replace draft tasks in plan.	`MANAGER`
`PUT`	`/api/v1/plans/{id}/assign`	Transition plan from `DRAFT` to `ACTIVE`; synthesize dates.	`MANAGER`
`POST`	`/api/v1/plans/{id}/certifications`	Add certification record to plan.	`MANAGER`
`POST`	`/api/v1/plans/{id}/connections`	Add connection contact to plan.	`MANAGER`
`GET`	`/api/v1/joinee/dashboard`	Joinee view: active plan, current week tasks, overdue counts.	`JOINEE`
`PATCH`	`/api/v1/tasks/{id}/status`	Update task status (`TO_DO`, `IN_PROGRESS`, `DONE`, `BLOCKED`).	`JOINEE`
`POST`	`/api/v1/certifications/{id}/proof`	Multipart upload for certification completion evidence.	`JOINEE`
`PATCH`	`/api/v1/connections/{id}/status`	Update connection milestone (`PENDING`, `SCHEDULED`, `MET`).	`JOINEE`

Method	Path	Description	Access Role
`GET`	`/api/v1/info-hub`	Category tree browse and full-text search.	Authenticated

8.2 Standard Envelope & Server-Side Scope Resolution

1. **JSON Envelope Structure**:

```
{
  "success": true,
  "data": {},
  "error": null,
  "timestamp": "2026-03-30T12:00:00Z"
}
```

2. **Error Format**:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "RESOURCE_FORBIDDEN",
    "message": "Authenticated manager is not mapped to the requested joinee resourc
    "details": []
  },
  "timestamp": "2026-03-30T12:00:00Z"
}
```

3. **Server-Side Context Resolution (Non-Negotiable)**: All endpoints that query or mutate manager-joinee hierarchies resolve tenancy using the authenticated `app_user.id` extracted from the verified JWT `sub` claim. Request headers, query strings, or body payloads containing `manager_id` or explicit authorization IDs are discarded. If a manager invokes `/api/v1/plans/{id}/assign`, the database queries `WHERE plan.id = :id AND plan.manager_id = :authUserId`. If no row matches, an HTTP 403 Forbidden is returned.
4. **Pagination**: List endpoints enforce standard parameters `?page=0&size=20`, max `size=100`.

5. **Versioning**: URL namespace versioning (`/api/v1/`). Breaking changes dictate `/api/v2/`.

---

9. Security and Privacy

9.1 Multi-Layered Enforcement

- **Layer 1 (Transport & Gateway)**: TLS 1.3 encryption mandatory. Reverse proxy strips unauthorized debug headers and enforces IP rate limits (100 req/min/IP).
- **Layer 2 (Authentication & Token Validation)**: Spring Security OAuth2 Resource Server validates cryptographic signature, expiration, and issuer against IdP JWKS endpoints.
- **Layer 3 (Server-Side Tenancy Interceptor)**: Domain-level ownership verification on every database mutation ensuring cross-tenant boundary isolation.
- **Layer 4 (Data Sanitization & S3 Sandboxing)**: Multi-part uploads scanned with ClamAV before write; files assigned randomized UUID storage keys with no direct public read URL (presigned GET URLs generated on demand with 15-minute expirations).

9.2 AI Data Boundary and Privacy

- PII scrubbing pipeline strips all personal identifiers prior to external model transit.
- Model provider integration contract mandates Zero Data Retention (ZDR) policy where submitted tokens are excluded from training corpuses.

---

10. Non-Functional Requirements

Category	Requirement Specification	Phase
Security	End-to-end TLS 1.3, complete JWT validation, zero anonymous API endpoints.	Phase 1
Security	Strict server-side tenancy resolution; untrusted client-supplied identity parameters rejected.	Phase 1
Security	ClamAV scanning and MIME-type verification on S3 proof uploads (10 MiB limit).	Phase 3

Category	Requirement Specification	Phase
Privacy	Zero PII transmission to external LLMs; payloads restricted to non-identifying operational context.	Phase 4
Privacy	LLM provider configured with zero data retention for model fine-tuning.	Phase 4
Performance	Non-AI transactional endpoints respond with p95 $< 500\text{ ms}$ at 100 concurrent active plans.	Phase 5
Performance	Web client initial dashboard render completes in p95 $< 2.0\text{ s}$ over corporate WAN.	Phase 5
Performance	AI generation pipeline completes in p95 $< 20\text{ s}$; hard timeout triggers fallback at 20 s.	Phase 4
Reliability	Degraded LLM provider transitions smoothly to manual entry without workflow blockage.	Phase 4
Availability	Core platform delivers $99.5%$ operational uptime during standard business hours.	Phase 5
Scalability	Relational schema and connection pools support ≥ 500 users and ≥ 100 concurrent plans.	Phase 1
Usability	Single-screen generate-edit-approve manager workflow; single-view joinee overdue visibility.	Phase 3
Accessibility	Full compliance with WCAG 2.1 Level AA (contrast, keyboard focus, screen-reader labels).	Phase 5
Compatibility	Layout rendering validated across Chrome, Edge, Firefox, Safari (viewport $\ge 768\text{ px}$).	Phase 5
Maintainability	Externalized prompt templates with golden regression tests; seed-managed Info Hub content.	Phase 2, 4
Observability	JSON application logs, Spring Actuator health metrics, LLM telemetry dashboards.	Phase 5
Cost	Redis SHA-256 context caching to eliminate duplicate model token burn.	Phase 4
Data Retention	Completed plans, logs, and files retained per policy [TBD - placeholder pending discovery].	Phase 5

11. Testing

11.1 Test Strategy and Levels

1. **Unit Testing (JUnit 5, Mockito, Vitest)**:
   - Date synthesis logic, metric formulas, and progress calculation edge cases (e.g., zero task division).
   - React component unit rendering, form validations, and keyboard navigation.
2. **Integration Testing (Spring Boot Test, Testcontainers)**:
   - PostgreSQL integration verifying Flyway scripts and full-text search indexes.
   - S3 upload pipeline with mock S3 (LocalStack) and infected file rejection.
3. **Contract and Security Testing**:
   - Cross-tenant security test suite attempting cross-manager modifications (must return 403).
   - OpenAPI schema validation between backend controllers and TypeScript client types.

11.2 Mandatory Release Gates

- **RELEASE GATE 1**: Cross-Tenant Security Suite must execute 100% green: no manager may access another manager's joinee data.
- **RELEASE GATE 2**: AI Fallback Assertion: Complete synthetic LLM outage simulation must result in seamless fallback to manual plan editing with zero data loss.
- **RELEASE GATE 3**: Schema Conformance Gate: LLM output parser must achieve $< 5%$ first-attempt JSON validation failure rate across the standard golden test suite.
- **RELEASE GATE 4**: WCAG 2.1 AA automated axe-core audit must pass with 0 critical or serious violations.

---

12. Deployment and Operations

12.1 CI/CD Pipeline

- **Continuous Integration**: On PR creation, GitHub Actions/GitLab CI runs Maven build, Vitest suite, checkstyle, Testcontainers integration tests, and golden prompt evaluations.
- **Artifact Generation**: Multi-stage Docker build packaging the Spring Boot application on Eclipse Temurin 21 Alpine image; static Vite bundle built and placed into Nginx/Gateway image.
- **Continuous Delivery**: Automated deploy to `'dev'` on merge; manual promotion gate to `'stage'` and `'prod'`.

12.2 Observability and Alerting Matrix

Signal / Metric	Source	Alert Condition	Severity	Action
LLM Schema Failure Rate	Prometheus / Micrometer	First-attempt failure $> 10%$ over 15m	Warning	Inspect prompt drift or provider model changes.
LLM Generation Latency	Spring Actuator Timer	p95 $> 18\text{ s}$ over 10m	Warning	Check provider API status; engage fallback if needed.
API Error Rate	Gateway Access Logs	5xx errors $> 1%$ over 5m	Critical	Inspect database connection pool and backend logs.
Cross-Tenant Auth Denials	Security Audit Log	> 5 events per minute	Critical	Investigate malicious traversal or token leakage.
Scheduled Job Execution	Spring Scheduler	Daily notification job fails or dies	High	Trigger manual runbook retry via actuator endpoint.

13. Technical Risks

ID	Risk	Impact	Mitigation
TR-RISK-01	LLM outputs non-conforming or malformed JSON payloads.	Medium	Enforce provider structured JSON mode; run Jackson schema validation; retry once with error context; failover to manual.
TR-RISK-02	External LLM API experiences rate limits, high latency, or total outage.	High	Enforce hard 20-second timeout; decouple plan management completely from AI availability.

ID	Risk	Impact	Mitigation
TR-RISK-03	Sensitive employee PII inadvertently transmitted to third-party model.	High	Strip user entities at service layer; permit only whitelisted context attributes in prompt builders.
TR-RISK-04	Cross-tenant data breach via client-manipulated IDs.	High	Enforce database query tenancy derived exclusively from verified JWT security context.
TR-RISK-05	Malicious file upload via certification proof submission.	High	Strict MIME-type checking, ClamAV container scanning, random key assignment in private S3 bucket.
TR-RISK-06	Information Hub full-text search performance degradation.	Medium	Utilize PostgreSQL generated `` `tsvector` `` columns with GIN indexes; cap payload limits.
TR-RISK-07	Duplicate prompt submissions cause runaway API cost.	Medium	SHA-256 context hashing cached in Redis with 7-day TTL; generation rate limiting per plan.

14. Assumptions, Dependencies, and Open Questions

14.1 Architectural Assumptions

1. PostgreSQL 16 `` `tsvector` `` is adequate for Information Hub search in v1 without requiring dedicated Elasticsearch/OpenSearch clusters.
2. The initial active concurrency (≤ 100 plans) does not warrant Quartz distributed clustering; Spring Scheduler with database-level advisory locking is sufficient.

14.2 System Dependencies

- **Corporate IdP**: Availability of Google Workspace / Azure AD OIDC metadata and JWKS endpoints.
- **Commercial LLM API**: API account provisioning, secret access keys, and SLA agreements.
- **SMTP Gateway**: Internal relay host credentials for transactional notifications.

- **S3 Storage**: Bucket provisioning and IAM role assignment.

14.3 Technical Open Questions

- What are the concrete data retention rules for `ai_run_log` and `audit_event` tables? [Placeholder pending discovery]
- What is the approved monthly dollar cap on external LLM token consumption? [Placeholder pending discovery]
- Will the deployment target be Amazon ECS, Kubernetes, or native virtual instances? [Placeholder pending discovery]

---

15. Appendix

15.1 Prompt Template and Output JSON Schema

Base System Prompt Template (`task_generation_v1.st`)

```
You are an expert technical onboarding assistant. Generate a structured onboarding plan
Context:
- Job Title: {job_title}
- Team / Department: {team}
- Plan Duration: {plan_duration_days} days
- Manager Notes / Directives: {manager_brief}

Rules:
1. Output MUST strictly match the provided JSON schema.
2. Generate between 5 and 15 tasks spread across weeks 1 to 4.
3. day_offset must be an integer between 0 and {plan_duration_days}.
4. Return ONLY valid JSON. No conversational preamble or postscript.
```

JSON Output Validation Schema (JSON Schema Draft-07)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "AITaskGenerationResponse",
```

```
    "type": "object",
    "required": ["tasks"],
    "additionalProperties": false,
    "properties": {
      "tasks": {
        "type": "array",
        "minItems": 1,
        "maxItems": 15,
        "items": {
          "type": "object",
          "required": ["title", "description", "category", "week_no", "day_offset", "pri
          "additionalProperties": false,
          "properties": {
            "title": {
              "type": "string",
              "minLength": 5,
              "maxLength": 100
            },
            "description": {
              "type": "string",
              "maxLength": 500
            },
            "category": {
              "type": "string",
              "enum": [
                "TECHNICAL_SETUP",
                "ACCESS_AND_TOOLS",
                "COMPLIANCE",
                "TEAM_AND_PROCESS",
                "ROLE_RAMP_UP",
                "ADMIN"
              ]
            },
            "week_no": {
              "type": "integer",
              "minimum": 1,
              "maximum": 4
            },
            "day_offset": {
              "type": "integer",
```

```
            "minimum": 0,
            "maximum": 30
          },
          "priority": {
            "type": "string",
            "enum": ["HIGH", "MEDIUM", "LOW"]
          },
          "subtasks": {
            "type": "array",
            "maxItems": 6,
            "items": {
              "type": "string",
              "maxLength": 100
            }
          }
        }
      }
    }
  }
```

15.2 Representative AI Run Log Entity

```
{
  "id": "7b8b4b1a-28e4-4d87-8d99-5ef861c8a14b",
  "plan_id": "c39d40a8-b64d-4467-bc18-2e061801f92e",
  "manager_id": "e81d7f72-69c5-4927-bdf2-f5e27a92df4c",
  "prompt_version": "v1.2",
  "model_identifier": "gpt-4o-mini",
  "input_brief": "Senior Backend Engineer joining Platform Payments. Focus on Java 21,
  "raw_output": {
    "tasks": [
      {
        "title": "Setup Local Java 21 & IDE Environment",
        "description": "Configure IntelliJ IDEA with internal checkstyle, JDK 21, and
        "category": "TECHNICAL_SETUP",
        "week_no": 1,
        "day_offset": 1,
        "priority": "HIGH",
```

```
          "subtasks": ["Install JDK 21", "Clone Payments repo", "Run test suite"]
        }
      ]
    },
    "validation_passed": true,
    "validation_errors": null,
    "prompt_tokens": 312,
    "completion_tokens": 145,
    "total_tokens": 457,
    "latency_ms": 1840,
    "is_accepted": true,
    "is_edited": false,
    "created_at": "2026-03-30T09:15:22Z"
  }
```


16. AI Boundary Statement

Artificial intelligence is used for exactly one capability in this product: expanding a manager's short free-text brief into a structured draft task list. It is not used to select certifications, to recommend which people a joinee should meet, to summarise progress, or to generate or answer questions over Information Hub content. Those are either manager-entered or static seeded content by deliberate design. The governing rule is that AI proposes and the manager approves. No AI-generated content becomes visible to a joinee without an explicit manager approval action, and no workflow in the product depends on the model being available. This boundary exists so that accountability for what a new joinee is told stays with a named human, so that the system remains predictable and auditable, and so that operating cost stays low and bounded. Any proposal to extend AI beyond this boundary requires a formal change request and re-approval by the decision audience.

17. AI Output Contract

The AI must return a JSON object containing a single tasks array. Each element must provide: title (string, 5 to 100 characters); description (string, up to 500 characters); category (one of TECHNICAL_SETUP, ACCESS_AND_TOOLS, COMPLIANCE, TEAM_AND_PROCESS, ROLE_RAMP_UP, ADMIN); week_no (integer, 1 to 4); day_offset (integer, 0 to 30, expressed as days from the joinee's start date); priority (HIGH, MEDIUM or LOW); and an optional subtasks array of up to six short strings. A maximum of 15 tasks may be returned in a single generation. The response is validated server-side against this schema before it reaches the manager. Malformed responses trigger exactly one retry with a corrective instruction; a second failure surfaces the manual entry form. The AI never returns due dates — only day offsets — because concrete dates are derived at assignment time from the joinee's actual start date. This keeps generated content reusable and prevents date errors from propagating from the model.

---

18. Information Hub Content Structure

Information Hub content is stored in an info_resource table with the fields: id, category, title, body_md (Markdown), contact_email, link_url, display_order, last_reviewed_date and is_active. It is populated from a versioned seed file applied through a database migration, so that corrections require a content edit and redeploy rather than a code change. The six launch categories are: IT and Access, covering the helpdesk contact, how to raise a support ticket, ticket SLA and VPN setup; Accounts and Tools, covering email, Slack, Jira, GitHub and who approves each access request; HR and Payroll, covering payroll contact, leave policy, holiday calendar and benefits; Workplace, covering office timings, seating, ID cards and reimbursement; Security and Compliance, covering the code of conduct, data handling policy and mandatory reading; and FAQ, a plain list of who to ask for what. Each category has a named owner in IT or HR who is accountable for accuracy. The displayed last-reviewed date makes staleness visible to joinees rather than hiding it.

---

19. Role and Permission Matrix

MANAGER may: create and edit joinee records within their own reporting line; create, edit and delete plans, tasks, certifications and connections for their own joinees only; trigger AI task generation;

approve and assign plans; view progress for their own joinees; and read the Information Hub. MANAGER may not: access any joinee outside their reporting line; edit Information Hub content; or modify a joinee's recorded task completions. JOINEE may: read their own active plan, certifications and connections; update the status of their own tasks, certifications and connections; upload their own certification proof; add notes to their own tasks; and read the Information Hub. JOINEE may not: view any other joinee's data; view draft plans; create, edit or delete tasks, certifications or connections; or invoke AI generation. All authorisation is enforced server-side on every request. Ownership is resolved from the authenticated token, never from an identifier supplied in the request body or query string.

---

20. Definition of MVP

The minimum viable product is Phases 0 through 4 inclusive: authentication with two roles and enforced ownership, the seeded Information Hub, full manual plan and task management, certifications, people to connect with, the joinee dashboard with progress and overdue visibility, the manager joinee list, and AI-assisted task drafting with a mandatory manager approval step and a working manual fallback. Notifications, the full audit trail, accessibility certification and observability dashboards are Phase 5 and are not required for the pilot. The MVP is considered viable when one manager can onboard one real joinee end to end using only this tool, and the joinee needs no email or verbal instruction to know what to do, what to complete, whom to meet, or where to get IT and HR help.

---

21. Measuring AI Quality

AI quality is measured by manager behaviour, not by subjective output review. Three metrics are tracked per generation and reported weekly. Acceptance rate is the share of generated tasks that survive to assignment without deletion, targeted above 70%. Edit rate is the share of assigned tasks whose fields were modified before assignment, targeted below 30%, where a high figure signals the prompt is producing content that is close but consistently wrong in a specific dimension. Regeneration rate is the number of times a manager re-runs generation for the same plan, where

anything above one repeatedly indicates the first output is unusable. Schema validation failure rate on first attempt is tracked separately as a reliability rather than a quality signal, targeted below 5%. A set of golden briefs with expected output characteristics is maintained as a regression suite, and every prompt version change is evaluated against it before release.

---

22. Out-of-Scope AI Uses and Why

Four AI capabilities were considered and deliberately excluded from v1. AI-recommended people to connect with was excluded because it requires an accurate org directory the organisation does not currently expose, and because an algorithm choosing whom a new employee should build relationships with raises fairness concerns that outweigh the convenience; the manager already knows the answer. AI-generated progress summaries were excluded because a completion percentage with an overdue count conveys the same information deterministically, at zero cost and with no risk of a misleading summary. An AI assistant answering joinee questions over company documents was excluded because it requires a curated corpus, a vector store and a security review of which documents a joinee may see; the static Information Hub answers the common questions with guaranteed accuracy. AI-suggested certifications were excluded because certification requirements are compliance-driven, and a hallucinated or omitted mandatory certification carries genuine regulatory risk. Each remains a candidate for a later release once v1 usage data justifies the additional cost and review burden.